



Compiler Principle ——Programming Language

Zhizheng Zhang
Southeast University



1. Programming Language Basics

- ❑ Static/Dynamic
- ❑ Environments/States
- ❑ Static Scope/Block Structure
- ❑ Explicit Access Control
- ❑ Dynamic Scope
- ❑ Parameters Passing Mechanisms
- ❑ Aliasing



1. 1 Static/Dynamic

□ Static vs. Dynamic

□ *Static Issues can be addressed during the compile time.*

✓ E.g., Scope of declarations in C or Java. Location of objects of a static class.

□ *Dynamic Issues are addressed at running time*

✓ E.g., Location of local variables.



1. 2 Environments/States

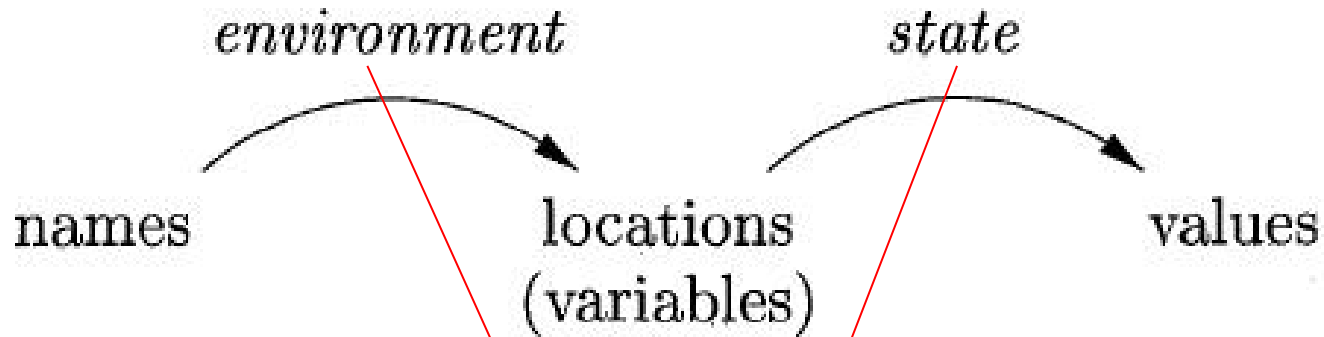


Figure 1.8: Two-stage mapping from names to values

Static /Dynamic



1. 3 Static Scopes/Block Structures

```
main() {  
    int a = 1;  
    int b = 1;  
    {  
        int b = 2;  
        {  
            int a = 3;  
            cout << a << b;  $B_3$   
        }  
        {  
            int b = 4;  
            cout << a << b;  $B_4$   
        }  
        cout << a << b;  
    }  
    cout << a << b;  
}
```

B_1

B_2

B_3

B_4

DECLARATION	SCOPE
int a = 1;	$B_1 - B_3$
int b = 1;	$B_1 - B_2$
int b = 2;	$B_2 - B_4$
int a = 3;	B_3
int b = 4;	B_4

Figure 1.10: Blocks in a C++ program



1. 4 Explicit Access Control

Public
Private
Protected

1. 5 Dynamic Scope

A use of a name x refers to the declaration of x in the most recently called procedure with such a declaration

E.g., macro expansion, method resolution in O-O Programming

```
#define a (x+1)

int x = 2;

void b() { int x = 1; printf("%d\n", a); }

void c() { printf("%d\n", a); }

void main() { b(); c(); }
```

Figure 1.12: A macro whose names must be scoped dynamically



1. 6 Parameter Passing

- *Call by Value*

e.g. `f(x) {x=x+1; print (x)};`
`main {y=1; f(y+1);}`

- *Call by Reference*

e.g. **array, pointer, object of all classes**

- *Call by Name*

e.g. **Macro definition**

1. 7 Aliasing

An interesting consequence of call-by-reference parameter passing. It is possible that two formal parameters can refer to the same location.

***E.g.,** Suppose a is an array belonging to a procedure p , and p calls another procedure $q(x, y)$ with a call $q(a, a)$.*

The important point is that if within q there is an assignment $x[10] = 2$, then the value of $y[10]$ also becomes 2.



2. Programming Language Design

- Artificial/formal Language
- Grammar
- Language Recognition
- Automata
- Grammar Construction

2. 1 Formal Language

□ Formal (Artificial) Language

- Vocabularies Table. *E.g., $\{a, b\}$*
- Sentence: *a sequence of lexemes, E.g. ab, baa .*
- Language: *a set of all sentences. E.g., $\{ab, baa\}$*

Note: not all sequences of lexemes are sentences, A sentence is a sequence grouped by constructions laws from lexemes.



OPERATION	DEFINITION AND NOTATION
<i>Union of L and M</i>	$L \cup M = \{s \mid s \text{ is in } L \text{ or } s \text{ is in } M\}$
<i>Concatenation of L and M</i>	$LM = \{st \mid s \text{ is in } L \text{ and } t \text{ is in } M\}$
<i>Kleene closure of L</i>	$L^* = \bigcup_{i=0}^{\infty} L^i$
<i>Positive closure of L</i>	$L^+ = \bigcup_{i=1}^{\infty} L^i$

L and M are sets of strings.



Example : Let L be the set of letters $\{A, B, \dots, Z, a, b, \dots, z\}$ and let D be the set of digits $\{0, 1, \dots, 9\}$.

- $L \cup D$
- LD
- $L(L \cup D)^*$
- L^4
- D^+



2.2 Grammar

□ Grammar I

● A Grammar *includes a collection of construction laws which are usually represented in the form of productions, a set of terminal symbols, and a set of non-terminal symbols, one of those non-terminal symbols is start symbol.*



□ Grammar II

● Example 1. $G=(T, N, P, S)$, where

$$T=\{a, b\},$$

$$N=\{A, B, S\},$$

$$S \in N,$$

$$P=\{S \rightarrow A|B,$$

$$A \rightarrow ab,$$

$$B \rightarrow baa$$

}

□ Grammar III

From grammars to languages, there is a one-to-one mapping.

The language of a grammar G is written as $L(G)$, which is the set of sequences of terminal symbols derived from the start symbol by replacing left sides of a production with its right side.



□ Grammar VI

Derivation: *Start with Start symbol, replacing an appear of a rule's left side with its right side.*

E.g., $S \rightarrow A, A \rightarrow ab,$
 $S \rightarrow B, B \rightarrow baa.$



2.3 Language Recognition

□ TASK

*Given a grammar G , and a sequence α of symbols, to check if $\alpha \in L(G)$, i.e., **if α can be derived from the start symbol of G .***



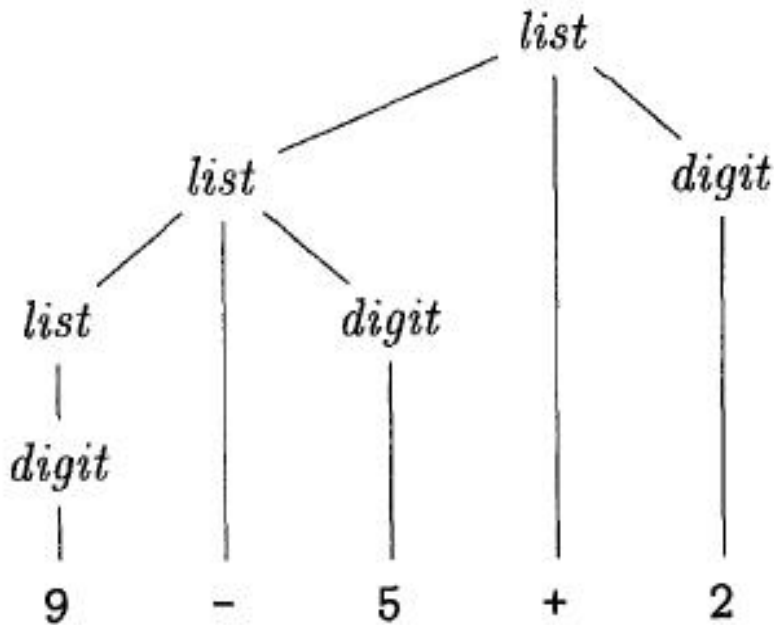
□ Example

$list \rightarrow list + digit$

$list \rightarrow list - digit$

$list \rightarrow digit$

$digit \rightarrow 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9$



list → **list** + digit
→ **list** - digit + digit
→ **digit** - digit + digit
→ 9 - **digit** + digit
→ 9 - 5 + **digit**
→ 9 - 5 + 2



□ Ambiguous Grammar

Two or more parse trees for some sentences

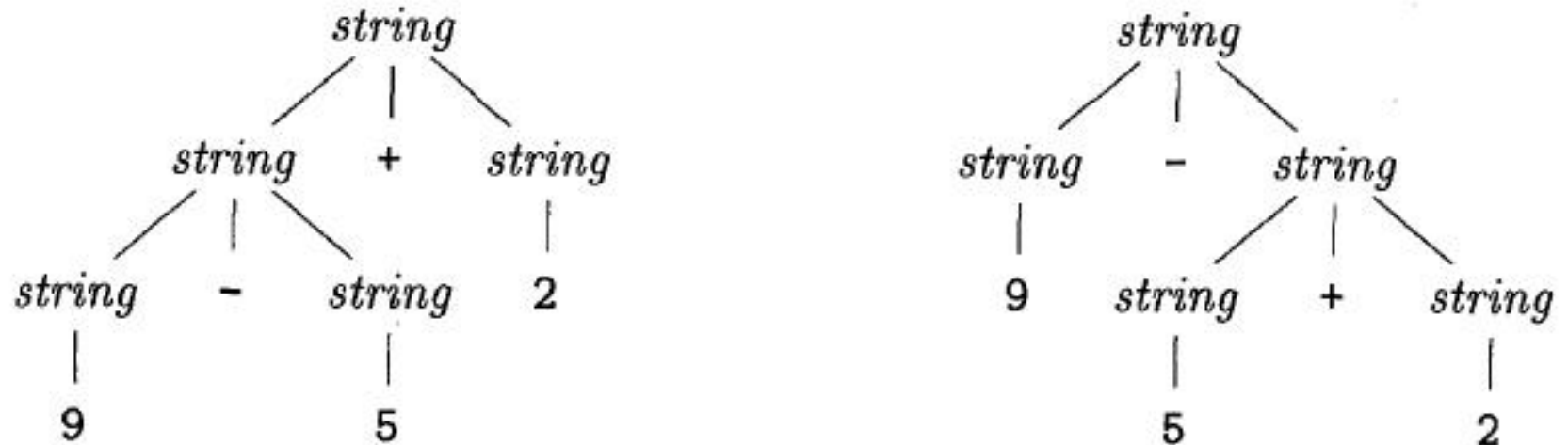


Figure 2.6: Two parse trees for 9-5+2

2.4 Automata

□ Grammar vs Automata

An automata of a grammar can be regarded as an algorithm of recognizing its language.

See the relationship between grammars and automata in the next page.



Hierarchy	Alias	Production form	Automation name
0-type	Grammar without limitation	$\alpha \rightarrow \beta$. where $\alpha \in (V_N \cup V_T)^* V_N^+ (V_N \cup V_T)^*$, $\beta \in (V_N \cup V_T)^*$	Turing Machine
1-type	Context-sensitive grammar	$\alpha A \beta \rightarrow \alpha \gamma \beta$. Where $A \in V_N$, $\alpha, \beta \in (V_N \cup V_T)^*$	Linear Bound Automation
2-type	Context-free grammar	$A \rightarrow \beta$. Where $A \in V_N$, $\beta \in (V_N \cup V_T)^*$.	Pushdown automation
3-type	Regular grammar	$A \rightarrow \alpha B$ or $A \rightarrow \alpha$. where $A, B \in V_N$, $\alpha \in V_T^*$	Finite automatio n

2.5 Construct a grammar for a language

——Some Examples & Skills

Method 1. Stepwise Dividing (逐步分解法)

- E.g, $L = \{a^i \mid i \geq 0\}$

1) If $i=0$ $L \rightarrow \varepsilon$

2) if $i \geq 1$ $a^i = aa^{i-1} = aa^j \ j \geq 0$
 $L \rightarrow aL$

3) The grammar is as following: $L \rightarrow aL \mid \varepsilon$

- E.g, $L = \{a^i b^j \mid i \geq 1, j \geq 0\}$

1) If $i > 1$ $a^i b^j = aa^{i-1} b^j = aa^k b^j \ k \geq 1, j \geq 0$ $L \rightarrow aL$

2) If $i=1$ $a^i b^j = ab^j$ $L \rightarrow aB$

3) $B \rightarrow bB \mid \varepsilon$

4) $L \rightarrow aB \mid aL$
 $B \rightarrow bB \mid \varepsilon$

- E.g, $L = \{i \mid i \bmod 5 = 0\}$

Firstbit	middlebits	Lastbit
----------	------------	---------

● For first bit

$L \rightarrow 1A|2A|3A|\dots|9A$

● For middlebit

$A \rightarrow 0A|1A|2A|\dots|9A|B$

● For lastbit

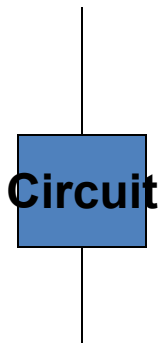
$B \rightarrow 0|5$

● $L \rightarrow 0|5$

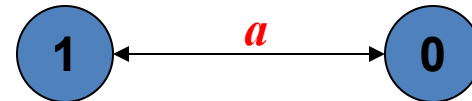
Method 2. Circuit Diagram (电路图法): odd and even problems

- E.g 1, $L = \{\omega \mid \omega \in a^*, \text{ and the number of } a \text{ in } \omega \text{ is odd}\}$

1) Imagine two states 0 and 1 in a circuit with one bit input



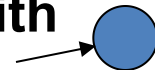
2) Two states can be transformed from each other with only one bit a input



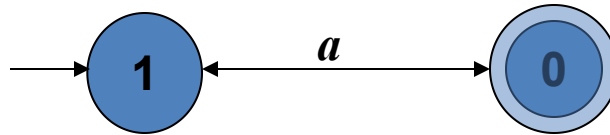
3) Decide the *start* state and *end* state.

We always assume that the state with full 0 bits is the end state. And we decide the start state according to the requirement in the problem. If we need a specific symbol with odd numbers of occurrences, we make the correspond bit 1, otherwise 0.

So in the above problem, we have the end state with
we have the start state with



4) Circuit diagram.



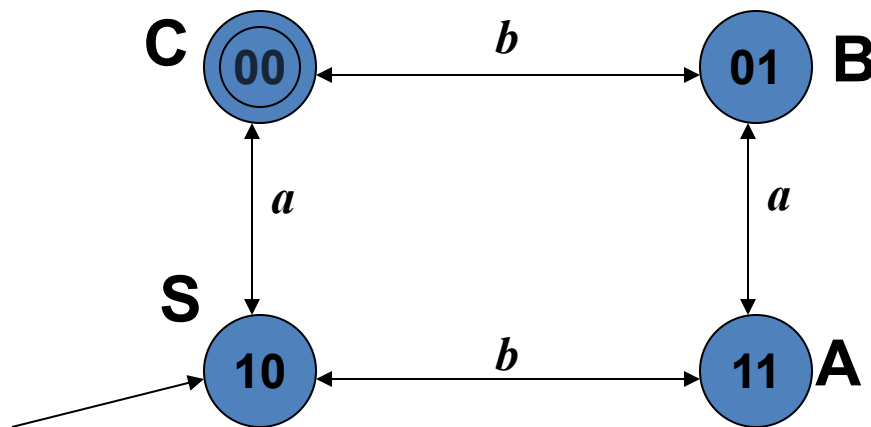
5) Grammar. According to the circuit diagram,
Write a non-terminal corresponding to a state. Let the start
state has start terminal.

$$A \rightarrow aB$$

$$B \rightarrow aA \mid \varepsilon$$

- E.g.2. $L = \{\omega \mid \omega \in (a, b)^*, \text{ and the number of } a \text{ in } \omega \text{ is odd, and that of } b \text{ is even}\}$

1) Construct the circuit diagram: **Two input bits and four states. Num of a is odd, num of b is even, so start state is 10**



2) Grammar

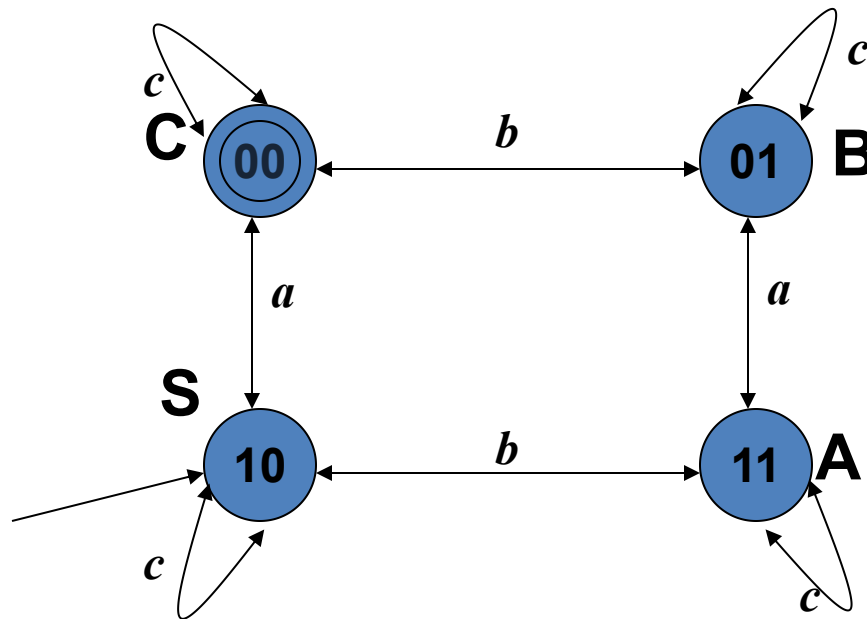
$$S \rightarrow aC \mid bA$$

$$A \rightarrow aB \mid bS$$

$$B \rightarrow aA \mid bC$$

$$C \rightarrow aS \mid bB \mid \varepsilon$$

- E.g.3. $L = \{\omega \mid \omega \in (a, b, c)^*, \text{ and the number of } a \text{ in } \omega \text{ is odd, and that of } b \text{ is even}\}$



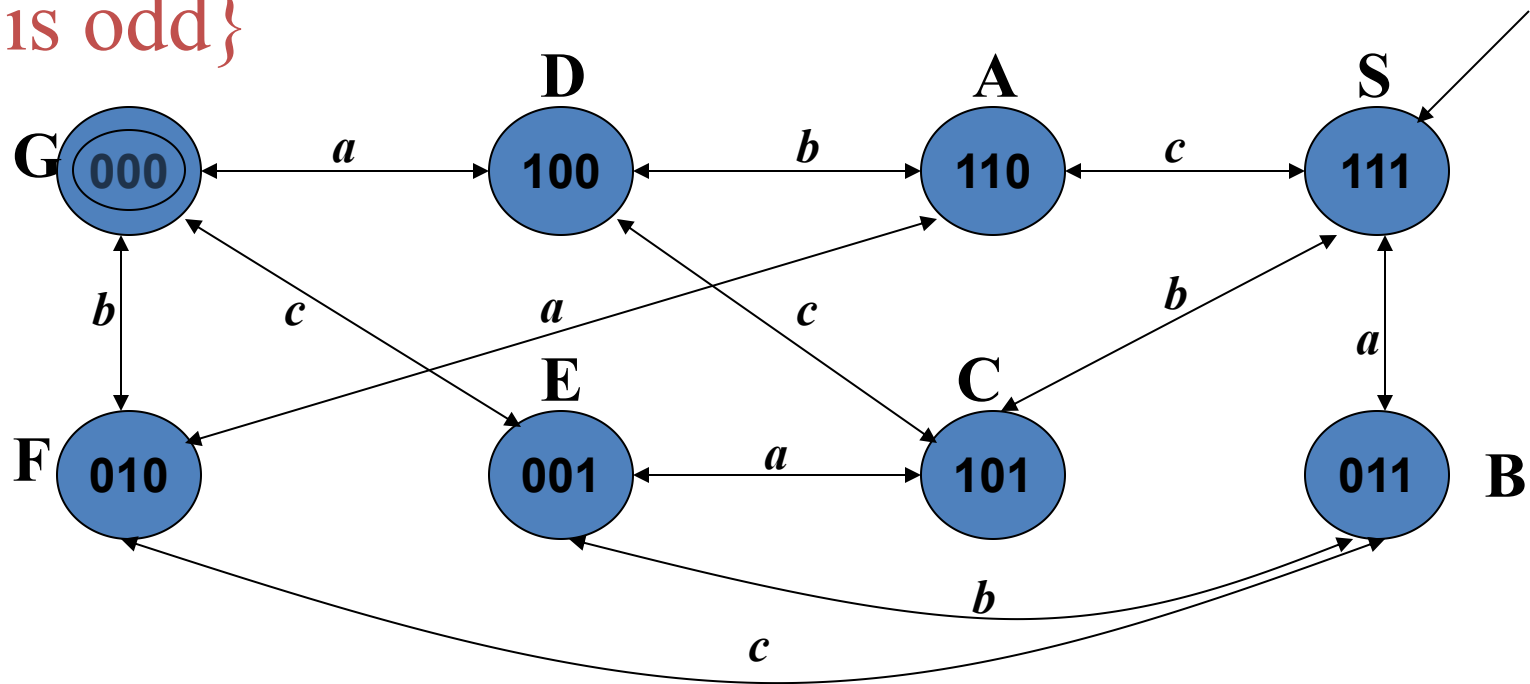
$$S \rightarrow aC \mid bA \mid cS$$

$$A \rightarrow aB \mid bS \mid cA$$

$$B \rightarrow aA \mid bC \mid cB$$

$$C \rightarrow aS \mid bB \mid cC \mid \varepsilon$$

- E.g.4. $L = \{\omega \mid \omega \in (a, b, c)^*, \text{ and the number of } a \text{ in } \omega \text{ is odd, and that of } b \text{ is odd, and that of } c \text{ is odd}\}$



Method 3. Equivalent to 2 sides (两边等价法)

- E.g. $L = \{\omega \mid \omega \in (0, 1)^*, \text{ and the number of 0 in } \omega \text{ is equal to that of 1}\}$

- 1) S can start with '0' or '1', so $S \rightarrow 0A \mid 1B$
- 2) A has '0' less 1 than '1', so $A \rightarrow 1S \mid 0AA$
- 3) B has '0' more 1 than '1', so $B \rightarrow 0S \mid 1BB$

4) Special cases: $S \rightarrow \varepsilon$

From above all, we have:

$$S \rightarrow 0A \mid 1B \mid \varepsilon$$

$$A \rightarrow 1S \mid 0AA$$

$$B \rightarrow 0S \mid 1BB$$

Method 4. Symmetrical method (对称法)

Embedded Grammar

- E.g.1. $L = \{a^i c b^i \mid i \geq 0\}$

- 1) Special case1: $S \rightarrow c$
- 2) Special Case2: $S \rightarrow a c b$
- 3) ...
- 4) ...
- 5) $S \rightarrow a S b \mid c$

- E.g.2. $L = \{a^m c b^n \mid i \geq 0\}$

- 1) Special case1: $S \rightarrow c$
- 2) Special Case2: $S \rightarrow a^m c b^n$
- 3) ...
- 4) ...
- 5) $S \rightarrow a^m S b^n \mid c$

- E.g.3. $L = \{ \omega c \omega^R \mid \omega \in (a, b)^* \}$

- 1) Special case1: $S \rightarrow c$

- 2) Special Case2: $S \rightarrow abcba$

- 3) ...

- 4) ...

- 5) $S \rightarrow aSa \mid bSb \mid c$

Method 5. Hybrid (状态转移图方法)

- E.g.1. $L = \{\text{能够被2整除的正整数的二进制数}\}$
- Exercise. $L = \{\text{能够被5整除的正整数的二进制数}\}$

提示:

1. 考虑余数变化规律, 构造状态转移图;
2. 将状态转移图转化为左(右)线性文法

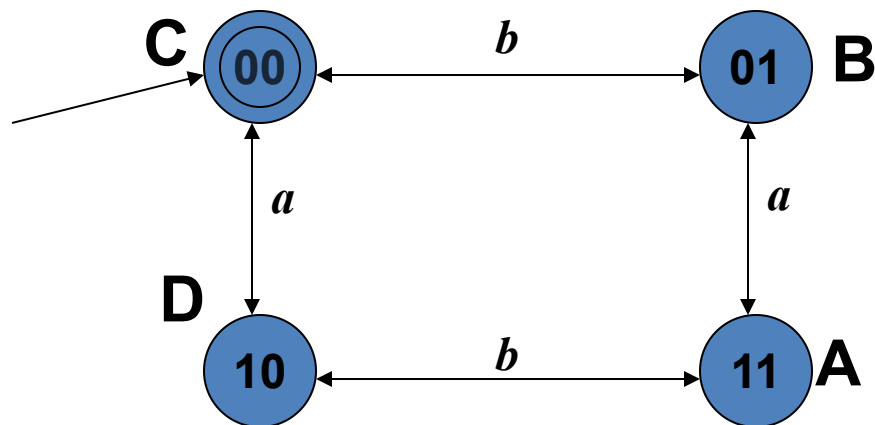
Method 6. Hybrid (混合法)

- E.g.1. $L = \{a^i b^j \mid i \geq j \geq 1\}$
- We can get $a^i b^j = a^{i-j} a^j b^j =$

- Then $S \rightarrow AB$
And $A \rightarrow aA|a$
And $B \rightarrow aBb|ab$
- Exercise. $L = \{\omega \mid \omega \in (a, b)^*, \omega \text{ starts with } a, \text{ ends with } b, \text{ and the number of } a \text{ is odd, the number of } b \text{ is odd}\}$

- Way 1
- $S \rightarrow aHb$

H is a string of a and b , in which the number of a is even, and the number of b is even too.



$$H \rightarrow C$$

$$C \rightarrow bB \mid aD \mid \varepsilon$$

$$A \rightarrow aB \mid bD$$

$$B \rightarrow aA \mid bC$$

$$D \rightarrow aC \mid bA$$

- Then

$$S \rightarrow aCb$$

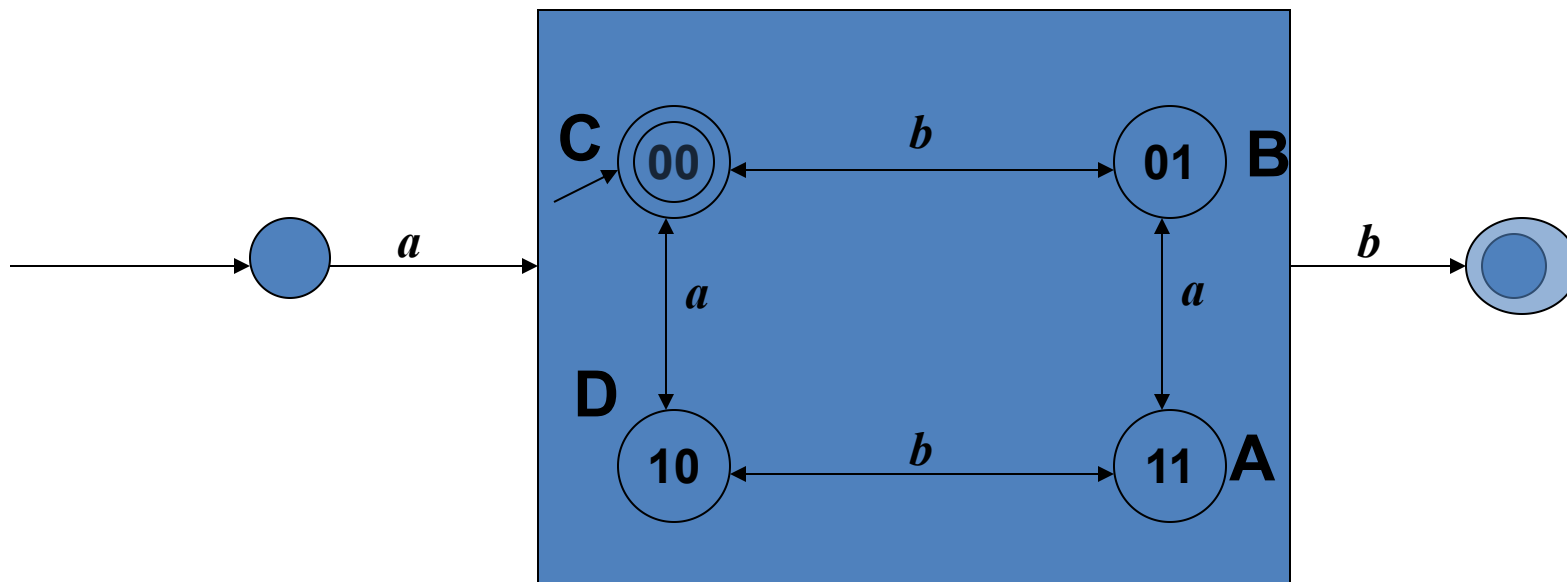
$$C \rightarrow bB \mid aD \mid \varepsilon$$

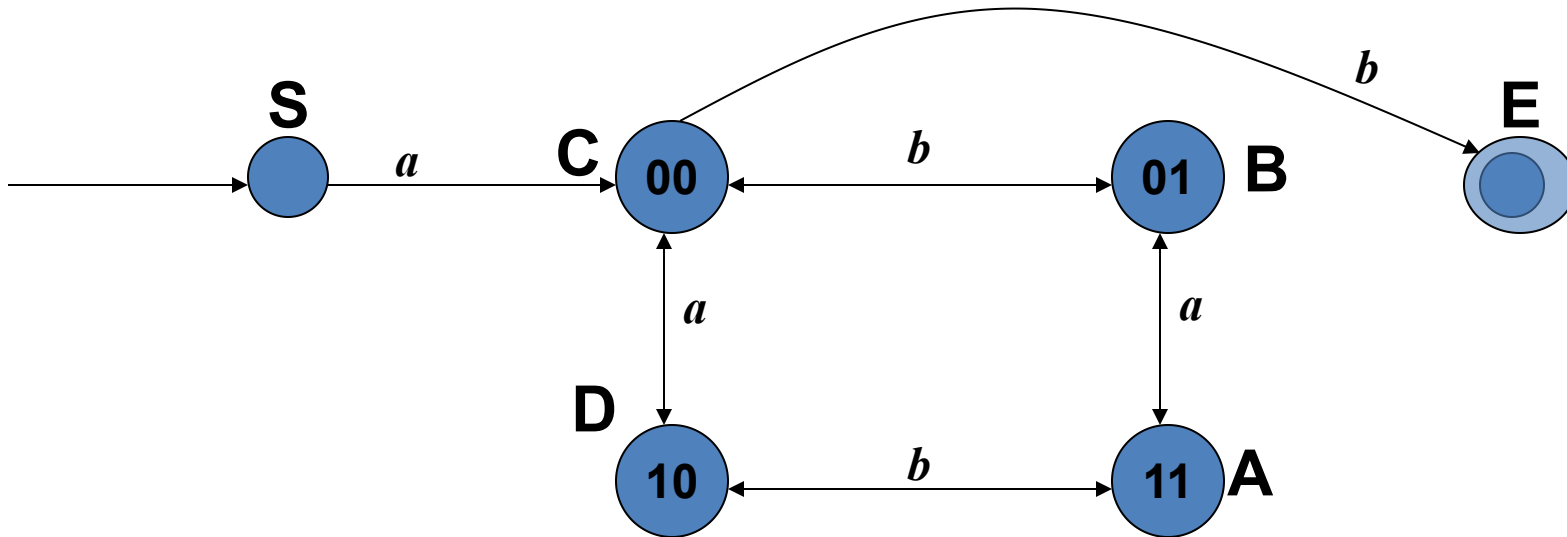
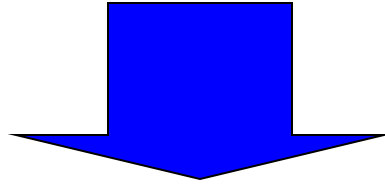
$$A \rightarrow aB \mid bD$$

$$B \rightarrow aA \mid bC$$

$$D \rightarrow aC \mid bA$$

- Way 2.





$$S \rightarrow aC$$

$$C \rightarrow bB \mid aD \mid bE$$

$$A \rightarrow aB \mid bD$$

$$B \rightarrow aA \mid bC$$

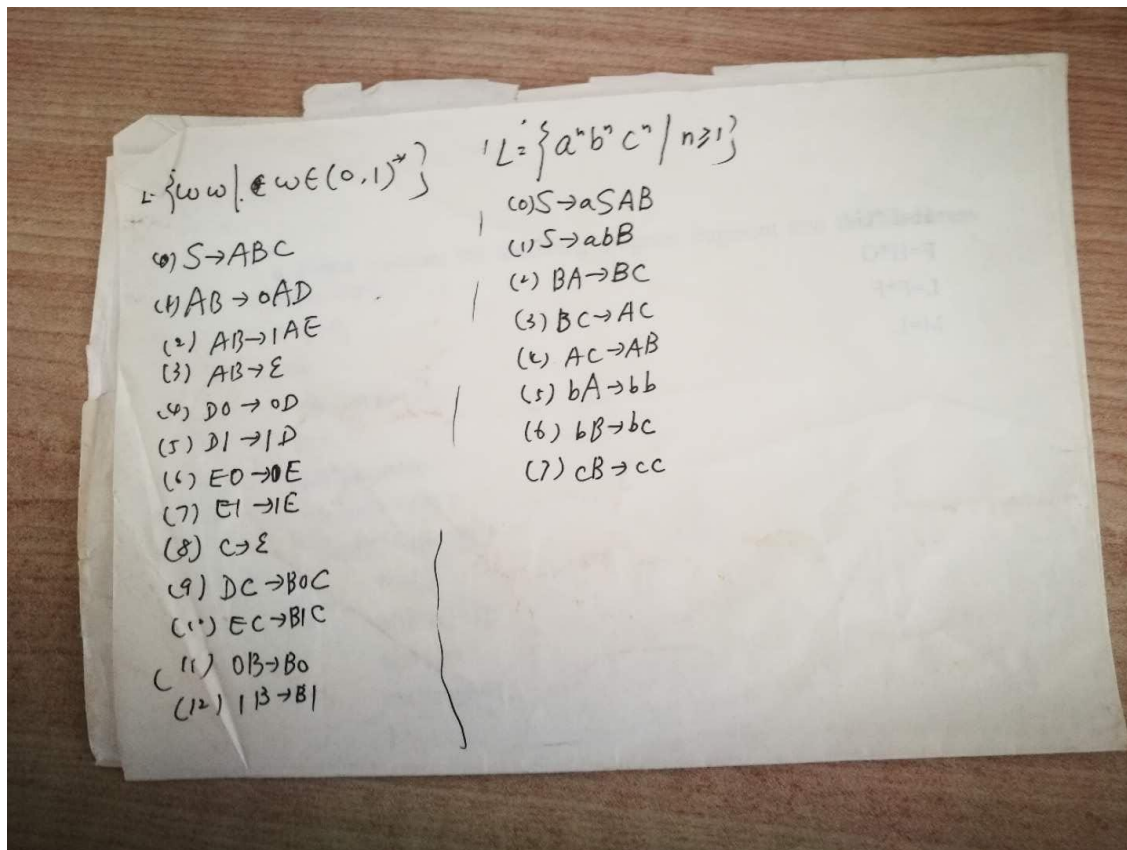
$$D \rightarrow aC \mid bA$$

$$E \rightarrow \varepsilon$$

Method 7. (非上下文无关文法)

- $L = \{ \omega\omega \mid \omega \in (a, b)^* \}$

- Exercise. $L = \{ a^m c^m b^m \}$



Exercises

- a) $\{\omega \mid \omega \in (a,b,c)^*$ and the numbers of a' s and b' s and c' s occurred in ω are **even**}
- b) $\{a^i b^j \mid i \geq (2j+1) \text{ and } j \geq 0\}$

- c) $\{\omega \mid \omega \in (a,b,c)^* \text{ and the numbers of } a' \text{ s and } b' \text{ s occurred in } \omega \text{ are } \mathbf{odd}\}$
- d) $\{a^i b^j \mid i \geq (j+1) \text{ and } j \geq 0\}$
- e) $\{\omega \mid \omega \in (a,b,c)^*, \omega \text{ is } \mathbf{lead by } a \text{ and the numbers of } a' \text{ s and } b' \text{ s occurred in } \omega \text{ are } \mathbf{even}\}$

- f) $\{a^{2i}b^{2j} \mid j \geq i \geq 1\}$
- g) $\{\omega \mid \omega \in (a,b,c)^* \text{ and } \omega \text{ starts with } a \text{ and ends with } b, \text{ the numbers of } a' \text{ s and } c' \text{ s occurred in } \omega \text{ are even}\}$
- h) $\{a^i b^j c^k \mid j \geq (i+k+1) \text{ and } i \geq 0, k \geq 1\}$

- i) $\{\omega \mid \omega \in (a,b,c)^*$ and **the numbers of a' s , b' s , c' s occurred in ω are all even**
- j) $\{\omega \mid \omega \in (a,b,c)^*$ and ω starts with a or b , ends with c , and the numbers of a' s and b' s and c' s occurred in ω are **even**
- k) $a^{2i-1}b^{2j-1}c^{2k-1} \ (i \geq 1, j \geq i+k, k \geq 1)$

END